

지오현

☎ 010-9973-7493 @jihkyuoo@naver.com

의료 AI·모빌리티 도메인에서 클라우드·온프레미스·B2B·B2C·어드민·데스크톱 앱 프로젝트를 주도한 프론트엔드 개발자입니다

문제의 본질과 우선순위를 먼저 정의하고, 난이도가 높은 병목 과제를 선제적으로 해결하며, 코드 일관성과 협업 생산성을 함께 끌어올렸습니다.

변경에 강한 구조, 명확한 책임 분리, 예측 가능한 상태 흐름을 갖춘 설계를 지향합니다. 이 관점을 바탕으로 신규 프로젝트 초기 구축 단계에서 보일러플레이트와 디자인시스템을 설계·개발하고, 아키텍처 기준과 컴포넌트/타입 설계 원칙, 코드리뷰 기준을 정립해 팀이 공통된 개발 방식으로 빠르게 실행할 수 있는 기반을 만들었습니다. 반복 되는 의사결정 비용과 코드 품질 편차를 줄이는 데 집중합니다.

혼자 앞서가기보다 팀 전체의 완주를 우선하며, 단기 대응보다 장기적인 유지보수성과 확장성을 기준으로 선행 과제를 끝까지 해결하는 페이스 메이커 역할을 지향합니다.

경력 4년 5개월



주식회사에이아이트릭스

2024.09 - 재직중 (1년 11개월) | 정규직

요약

2024.09 - 2026.04

VDOC의 의료 AI 제품군 3개 라인(PRO PC · PRO WEB · VDOC)의 프론트엔드를 주도적으로 개발하며, 제품별 핵심 아키텍처 설계와 함께 보일러플레이트·디자인시스템·인체도 자산을 팀 표준으로 정립했습니다.

VDOC PRO PC - 의료진용 진료 데스크톱 앱

2024.09 - 2026.04

병원에 공급되는 의료진용 진료 보조 데스크톱 앱입니다. 환자가 VDOC과 VDOC PRO WEB을 통해 입력한 문진 데이터가 실시간으로 전달되어 진료 현장에서 즉시 활용되며, 환자 문진부터 EMR 차팅까지 이어지는 진료 파이프라인에서 의료진이 실제로 진료를 보는 화면을 담당합니다.

[주요 작업]

1. Electron 자동 업데이트 — 클라우드 / 온프레미스 이중 체계

병원마다 데스크톱 앱을 방문 설치·업데이트하던 구조를 개선해, S3(클라우드)와 내부 서버(온프레미스)를 업데이트 피드로 분기하는 자동 업데이트 체계 구축

- 부팅 시 electron-updater로 자동 버전 체크 후 다운로드·설치

- 클라우드 병원은 electron-builder publish provider s3, 온프레미스 병원은 provider generic으로 내부 서버 URL 분기
- Windows(NSIS)와 macOS(DMG) 각각 빌드 타겟 대응
- 결과: 클라우드는 S3 업로드 한 번, 온프레미스는 내부 서버 업로드 한 번으로 다수 PC 일괄 자동 업데이트

2. 실시간 환자 상태 소켓 알림과 네이티브 배지

STOMP 소켓 이벤트를 IPC로 메인 프로세스에 넘겨 네이티브 Notification과 배지로 전달하는 의료진 알림 시스템 구축

- STOMP 소켓으로 환자 상태 변경 이벤트 수신 (5초 간격 재연결, heartbeat, 최대 10회 재시도)
- STOMP 소켓 이벤트를 전용 IPC 채널을 통해 메인 프로세스로 전달, Electron 네이티브 Notification 표시
- 알림 클릭 시 창 복원과 해당 진료 탭 자동 포커싱까지 연결
- 단발성 알림 누락 방지를 위한 앱 오버레이 배지 카운트 구현

3. 치료가이드 템플릿 시스템

CKEditor 기반 WYSIWYG 에디터와 커스텀 비디오 플레이어로 의료진이 직접 치료가이드를 작성·발송할 수 있는 시스템 구축

- CKEditor 기반 마크다운 툴바 에디터 — 노션 스타일로 마크다운 문법이 바로 텍스트에 적용되어 실제 환자에게 보여지는 화면 그대로 편집

- 치료 영상 삽입을 위한 커스텀 비디오 플레이어 (권장/비권장 상태 오버레이 배지 지원)
- 템플릿 CRUD, 카테고리 폴더, 검색 무한스크롤, 미리보기, 발송 흐름을 하나의 세트로 통합

4. 개발 모드 — 서버 프로파일 동적 전환

빌드 분기 없이 단일 앱으로 클라우드·온프레미스 여러 병원 환경을 QA할 수 있는 서버 프로파일 시스템 구축

- 서버 프로파일 3종 정의: publish(상용), 온프레미스 테스트, dev
- 각 프로파일이 API URL과 WebSocket URL을 한 세트로 묶음
- 로그인 화면 드롭다운에서 즉시 전환, localStorage에 저장되어 다음 실행 시 유지

[기타]

- 실시간 환자 상태 동기화
- OCR 이미지 뷰어(줌 인/아웃), AI 차트 요약, AI 추천 의증, EMR 차팅 복사
- 인체도 표시
- STT 녹음 요약본 생성과 대화 전문 표시
- 환자 관리(등록·수정·검색 모달, 무한스크롤 테이블)
- 권한별 라우팅 접근 제어와 사용자 액션 감사 로그(엑셀 다운로드)
- 비밀번호 실패 카운팅과 초기화, 계정 CRUD
- 미니 모드(컴팩트 진료 화면)
- i18n(한/영/일) 및 언어 우선도 설정

[Skills]

- Base: Electron, React, Typescript, Vite
- 자동 업데이트: electron-builder, electron-updater
- 상태 관리: Zustand
- 서버 상태: TanStack Query
- 실시간 통신: STOMP (@stomp/stompjs)
- Form: React Hook Form, Zod

- 인체도: D3.js
- MD 에디터: CKEditor
- Style: Emotion
- 국제화: i18n

VDOC PRO WEB - 환자용 문진 서비스

2025.03 - 2026.03

병원 내원 환자가 진료 전에 AI와 대화하며 문진을 진행하는 병원별 맞춤형 웹 서비스입니다. 환자가 입력한 증상과 답변은 VDOC PRO PC로 실시간 전달되어, 환자에서 의료진으로 이어지는 진료 파이프라인의 시작점을 담당합니다.

[주요 작업]

1. 서버드러본 UI 기반 확장형 문진 아키텍처

문진 구성·프로그레스·플레이스홀더·방문사유 흐름·커스텀 질문을 서버 스키마가 제어하도록 설계해, 병원·진료과별 커스텀을 프론트 배포 없이 런타임에 반영

- UI 동작 11개 이상과 질문 스키마 19여 개 필드를 서버가 제어
- 1차·3차 병원 티어 분기도 서버 스키마 기반으로 런타임 반영
- 기능 추가 시마다 프론트 배포가 따라가야 했던 구조적 한계 해결

2. 문진 시스템 V2 — 상태 기반에서 라우트 기반으로 재설계

단일 상태로 제어되던 V1을 라우트 단위로 쪼개 브라우저 히스토리와 동기화하고, 더티 체크로 AI 질문 컨텍스트까지 보존하는 구조로 전면 재설계

- V1 한계: 모든 문진 페이지가 단일 상태, 브라우저 뒤로가기 누르면 전체 문진 유실, UI 백버튼도 현재 답변 유실
- V2 구조: 라우트 단위로 페이지 분리 + 시퀀스 번호를 라우트 파라미터로 동적 렌더링 + 브라우저 히스토리와 동기화
- 문진 항목 타입 동적 렌더링: AI 텍스트 질문, 통증 점수, 커스텀 질문 12종
- 더티 체크 기반 답변 보존: 답변이 변경되지 않은 경우 이전 질문 컨텍스트를 Map에 저장해 재방문 시 그대로 복원

[설계 판단]

답변 보존이 중요한 이유는 AI 문진이 이전 답변 컨텍스트를 기반으로 새 질문을 생성하기 때문입니다. 답변이 같더라도 AI는 매번 다른 질문을 던질 수 있는데, 답변이 변경되지 않은 경우 기존 질문까지 컨텍스트로 보장해줌으로써 뒤로 갔다 돌아오는 것만으로 문진 흐름이 달라지는 현상을 막았습니다.

3. 소켓과 폴링 병렬 실행으로 AI 문진 끊김 제거

다음 문진 조회에서 소켓의 끊김 취약성과 폴링의 지연 딜레마를, 두 방식 병렬 실행 + 먼저 도착한 응답 채택 구조로 해결

- 폴링: 1초 주기, 최대 50회 자가복구
- 소켓: 5초 간격 재연결, heartbeat, 최대 10회 재시도
- 답변은 Map 컨텍스트에 저장해 폴링 머지 시 덮어쓰지 않도록 설계

4. 문진 중 의도치 않은 이탈 방지

TanStack Router useBlocker로 문진 외부 이동을 가로채 컨펌 모달로 의도 확인하는 구조

- 문진 내부에서는 자연스러운 뒤로/앞으로 이동 유지
- 문진 플로우 자체를 이탈할 때만 명시적 컨펌 모달 노출

[기타]

- FSD 아키텍처 기반 프로젝트 초기 구성과 라우팅 보호 설정
- 의료진별 맞춤 서비스 플로우 분기와 브라우저 히스토리 블로킹
- 로딩 처리 고도화(최소 표시 시간 보장, 짧은 로딩은 즉시 통과)
- 의사별 커스텀 고정 문진 체계
- 서류 이미지 업로드 후 OCR 처리 결과를 VDOC PRO PC로 전달하는 파이프라인
- Playwright를 통한 AI 문진 검증
- i18n(한/영/일) 및 언어 우선도 설정

[Skills]

- Base: React, Typescript, Vite
- 상태 관리: Zustand
- 서버 상태: TanStack Query
- 라우터: TanStack Router
- 실시간 통신: STOMP (@stomp/stompjs)
- Form: React Hook Form, Zod
- 인체도: D3.js
- Style: Emotion
- 국제화: i18n

VDOC

2025.05 - 2025.11

<https://ai.vdoc.kr/>

의료 특화 AI에게 증상을 입력하면 가능한 원인 분석과 건강 리포트를 받아볼 수 있는 B2C 웹 서비스입니다. 서비스 내 병원 접수를 통해 환자의 문진 데이터가 VDOC PRO PC로 전달되어 진료 현장까지 이어집니다. 초기 단계부터 설계·구축해 핵심 사용자 여정을 end-to-end로 완성했고, B2C 서비스 론칭을 가능하게 하는 프론트엔드 기반을 마련했습니다.

[주요 작업]

1. 비회원에서 회원으로의 자동 마이그레이션과 팝업 인증

게스트가 핵심 기능을 먼저 써보다 인증 지점에서 이탈하는 문제를, 팝업 인증 + 데이터 자동 마이그레이션 + 화면 복귀 구조로 해결

- window.open과 postMessage 기반 팝업 인증으로 현재 화면을 유지한 채 비동기 로그인 처리
- 로그인 성공 시 게스트를 회원 계정으로 자동 마이그레이션
- 별도의 리다이렉트 스토어로 원래 보던 화면에 무중단 복귀 — 가입 전 경험과 회원 전환 연속성 확보

2. 멀티 지도 프로바이더를 전략 패턴으로 추상화

Naver와 Google을 공통 MapEntity 인터페이스로 추상화하여 프로바이더 추가 및 유지보수가 동일 규격 위에서 이뤄지는 구조 설계

- 두 프로바이더 필요 배경: Naver는 한국 사용자 친화, Google은 해외 서비스 확장과 해외 거주자 대상 병원 접수 대응
- 전략 패턴 기반 MapEntity 인터페이스: 지도 생성, 마커 생성, 중심 이동, 줌 변경 등 7개 공통 메서드로 Naver·Google 구현체

통일

- 마커 렌더링 부하 제어: 줌 레벨과 뷰포트 기준으로 표시 마커 수 제한, 위치 변경 시 새로 그리고 기존 마커 정리
- 지도를 라우트 바탕에 두고 리스트·상세·접수 화면을 Outlet 레이어로 띄워 페이지 이동 시 지도 재렌더 방지
- 지도 현재 위치를 주소의 쿼리 파라미터와 동기화 — 라우트 변경 후 위치 복원 + URL을 통한 직접 위치 전달 확장성 확보

[설계 판단]

지도 페이지 구조에서 가장 경계한 것은 페이지 이동 때마다 지도가 다시 그려지는 비용이었습니다. 지도를 라우트 바탕 레이어로 고정하고 페이지들을 Outlet으로 올리는 구조와 주소 쿼리 파라미터 동기화를 조합해, 사용자가 보던 지도 위치를 잃지 않으면서도 병원 위치 공유 같은 확장 가능성까지 함께 열어줬습니다.

3. 다중 OAuth와 토큰 리프레시 큐, 다중탭 세션 동기화

3종 OAuth를 단일 추상화로 통합하고 동시 401 중복 요청을 failedQueue로 제거, 다중탭 세션을 localStorage 이벤트로 동기화

- OAuth 3종(카카오, 애플, PASS)을 단일 AuthProvider 추상화로 통합
- 동시 401 발생 시 failedQueue 기반 토큰 리프레시 큐 패턴으로 중복 요청 차단
- localStorage 이벤트 + Zustand persist로 다중탭 세션 동기화 — 한 탭 로그인/로그아웃이 전체 탭에 즉시 반영

4. 한국어 키 기반 i18n 구조와 언어 우선도 설계

영어 코드 대신 한국어 텍스트를 키로 사용하고 스캐너 스크립트로 동기화를 자동화, 언어 설정 우선도 체계 구축

- 키 네이밍을 영어 코드가 아닌 한국어 원문으로 사용 — 화면과 코드 탐색 용이, 중복 문구 단일 키 공유, 번역 작업 부담 경감
- 스캐너 스크립트로 코드베이스와 번역본 자동 동기화 (코드에 있는 키는 자동 추가, 코드에서 사라진 키는 자동 삭제)
- 언어 우선도: 로컬스토리지(URL의 lang 쿼리 파라미터나 사용자 언어 변경으로 기록됨) > 브라우저 언어
- 지원 언어(한국어, 영어, 일본어) 밖이면 영어로 폴백
- 사용자가 UI나 URL로 명시적으로 선택한 언어는 로컬스토리지에 기록해 브라우저 언어와 무관하게 유지

[기타]

- 약관 동의, 회원가입, 회원 탈퇴 플로우
- 다중 프로필 관리(추가·수정·삭제, 메인 프로필 스왑, 비회원 프로필)
- PHR 시스템(병원 방문과 약국 처방 기록, AI 요약 폴링)
- 병원 접수 이미지 업로드(VDOC PRO PC 파이프라인 연결)
- 문진 채팅과 지도, 병원 상세, 접수 화면 사이의 끊김 없는 레이어 처리
- Drawer 사이드 메뉴 구성, 뷰포트 모바일 최적화
- Vitest와 MSW 테스트 인프라 셋업
- i18n(한/영/일)과 지도 언어 반영, 언어 우선도 설정, 언어 변경 기능

[Skills]

- Base: React, Typescript, Vite
- 상태 관리: Zustand
- 서버 상태: TanStack Query
- 라우터: TanStack Router
- 실시간 통신: STOMP (@stomp/stompjs)
- Form: React Hook Form, Zod
- 인체도: D3.js

- Style: Emotion
- 지도: Google Maps, Naver Maps
- 인증: 카카오, Apple, Pass
- 국제화: i18n

인체도 구현

2024.09 - 2024.10

환자가 증상을 상세히 표현하고 의료진이 진료할 때 사용되는 인체도는 VDOC PRO PC, PRO WEB, VDOC 세 제품 모두에 필요했습니다. 복잡한 인체도가 여러 제품에 반복해서 들어가도 유연하게 붙을 수 있는 구조로 설계하고, 이미지 에셋 경량화로 빌드 용량까지 크게 절감했습니다.

- 코어 로직 추상화: 세 제품이 동일한 인체도 설계를 공유, 각 제품의 비즈니스 로직과 맞닿는 접점만 독립적으로 구현
- React Context 스코프 제한: 인체도 영역 내부에서만 유효하도록 설계해 전역 컨텍스트의 사이드이펙트 방지 (줌, 부위 클릭, 오버레이 표시 등 제어용 컨텍스트)
- 진료과별 분기 로직과 내부 매핑: 진료과-인체 부위 매핑, 모든 인체 부위에 서버·AI 소통용 고유 ID 매핑 — 환자가 선택한 부위 정보가 그대로 문진과 의료진 화면까지 이어짐
- 빌드 용량 절감: VDOC PRO PC 기준 21,410 kB → 2,959 kB (약 86% 감소), 다수의 인체 부위 이미지 에셋 경량화 결과

[설계 판단]

초기에는 VDOC PRO PC 하나만 존재했지만, 복잡한 인체도가 추후 다른 제품에도 들어갈 가능성을 일찍 판단해 처음부터 추상화에 투자했습니다. 코어 로직이 추상화되어 있지 않았다면 각 프로젝트별로 복잡한 인체도가 서로 다른 설계로 구현됐을 것이고, 유지보수 비용이 기하급수적으로 늘어났을 부분입니다.

디자인 시스템 구축 및 관리

2024.12 - 2025.12

VDOC 모든 프로젝트에서 사용되는 디자인시스템입니다.

초기 단계에서 컴포넌트 코드 구조·타입 규칙·스토리북 작성 기준이 정해지지 않으면, 컴포넌트가 늘어날수록 구현 방식이 달라져 재사용성과 유지보수성이 빠르게 떨어지기 때문에, 동일 컨벤션으로 확장할 수 있는 기반을 마련하여 컴포넌트 추가 시 품질 편차를 줄였습니다.

[Work]

- 프로젝트 초기 구성
- 컴포넌트 아키텍처 정립
- 스토리북, 유닛 테스트, 통합 테스트 도입
- Popup(비동기 레이어), Button, Input, TextArea, Badge, Switch, Provider, Toast, Table, Checkbox 개발

[Skills]

- Base: React, Typescript, Vite
- 상태 관리: Zustand
- Style: Emotion
- Test: vitest, storybook

웹 보일러플레이트 구축

2025.09 - 2025.09

프로젝트마다 초기 세팅과 폴더 구조, 상태 관리 방식, 공통 유틸 배치 기준이 제각각이어서 신규 프로젝트를 시작할 때마다 동일한 아키텍처 결정을 반복해야 했고, 이로 인해 착수 속도와 코드 일관성이 흔들릴 수 있는 상황이었습니다.
보일러플레이트와 아키텍처 표준을 정립하면서 신규 프로젝트가 공통 구조에서 빠르게 시작될 수 있게 되었고, 프로젝트 간 코드 구성 방식과 협업 기준의 일관성이 높아졌습니다.

[Work]

- 프로젝트 초기 구성
- FSD 폴더 아키텍처
- 초기 프론트엔드 구축 리소스 크게 절감
- 프론트엔드 개발환경 통일
- PR Template 구성 및 프론트엔드 컨벤션 정립

[Skills]

- Base: React, Typescript, Vite
- 상태 관리: Zustand
- 서버 상태: TanStack Query
- 라우터: TanStack Router
- Form: React Hook Form, Zod
- Style: Emotion



주식회사더스윙(TheSwingCO.,LTD)

2022.03 - 2024.09 (2년 7개월) | 정규직

[백오피스 어드민] 스왑 운영 관리 어드민

2024.05 - 2024.08

스왑 서비스를 운영 관리하는 어드민을 개발하였습니다.

Work

- 기존 스왑 관리 체계 전체 리뉴얼
- Map API 사용(스왑 기기 관제 GPS 지도)
- 엑셀 다운로드

Skills

- Base: React, Typescript, Vite
- Style: Antd, Styled-component
- 지도: Naver Maps API
- Mock: MSW

[웹사이트] 스윙 셔틀 (아놀자 x 스윙) 서비스

2024.04 - 2024.07

야놀자와 프로모션으로 진행하는 공항 동행 이동 서비스를 개발하였습니다.

Work

- 경유지 탑승 예약 시스템 개발
- 토스 결제 연동
- 국제화 작업을 위해 구글 스프레드 시트와 연동하여 자동화 프로세스 구축
- 맵 API 사용(픽업지 검색 지도)
- 타임존 대응(해외 예약 시 한국 타임존 기준으로 시간 표시)
- 반응형 레이아웃(데스크톱, 모바일) 및 Cross browsing

Skills

- Base: React, Typescript, tanstack-query, Vite
- Style: Styled-component
- 상태 관리: Recoil
- Form: React-hook-form
- 지도: Naver Maps API
- 결제 : 토스 결제
- 국제화: i18n
- Mock: MSW

[웹사이트] 스왑 서비스

2024.07 - 2024.08

<https://swapswap.kr/>

전기 자전거 구독/구매 서비스인 스왑을 개발하였습니다.

Work

- 신규 기종 추가
- 홈 화면 리뉴얼
- 프로모션 페이지 개발
- 반응형 레이아웃(데스크톱, 모바일) 및 Cross browsing

Skills

- Base: React, Typescript, Vite
- Style: Mantine, tailwindcss
- 지도: Naver Maps API
- 결제 : 이니시스

[웹뷰 & 웹사이트] 스윙택시 콜밴 서비스

2023.07 - 2024.07

<https://swing.taxi/>

(현재 사이트에 서비스 기능들은 빠지고 브릿지 페이지로 바뀌었습니다)

고객을 원하는 목적지까지 이동시켜주는 차량 콜밴 서비스인 스윙택시를 개발하였습니다.

Work

- 택시 콜밴 예약 시스템 개발
- 골프장 왕복 운행 예약 시스템 개발
- H3-js 사용(서비스 지역 Validation)
- 맵 API 사용(픽업지 검색 지도)
- 타임존 대응(해외 예약 시 한국 타임존 기준으로 시간 표시)
- PASS 인증
- 채널톡 연동
- 국제화(국문, 영문)
- 예약내역 인피니티 스크롤 조회
- 반응형 레이아웃(데스크톱, 모바일) 및 Cross browsing

Skills

- Base: React, Typescript, Vite
- Style: Styled-component
- 상태 관리: React Context API, Recoil
- 지도: Naver Maps API
- 국제화: i18n
- Mock: MSW

[백오피스 어드민] 스윙바이크 운영 관리 어드민

2023.04 - 2023.07

B2B 업체를 대상으로하는 스윙바이크의 운영 어드민을 개발하였습니다.

Work

- 기기 관리 기능 개발
- 사용자 관리 기능 개발
- 스윙바이크 어드민 비즈니스 플로우 기획 등 PM역할 수행
- GraphQL의 스키마 관리를 위한 백엔드 스키마 연동 및 gql 쿼리 자동 생성
- 엑셀 다운로드

Skills

- Base: React, Typescript, GraphQL, Apollo, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API
- Mock: MSW

[백오피스 어드민] 자산 & 파트너 & 계약 & 정산 관리 어드민

2023.02 - 2023.07

B2B 파트너 관리 및 정산을 위한 어드민을 개발하였습니다.

Work

- 관리 체계 및 화면 구성 기획
- 계약서 관리 시스템 개발
- 정산 결재 시스템 개발

- 정산 결과 PDF 출력 및 이메일 발송 기능 개발
- 엑셀 다운로드

Skills

- Base: React, Typescript, react-query, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API
- Mock: MSW

[웹뷰] 스윙 앱

2023.03 - 2023.03

스윙 앱을 버전업 하면서 추가된 웹뷰인 헤택 페이지를 개발하였습니다.

Work

- 이벤트 페이지 개발(네이티브와 브릿지 통신)
- 국제화(영문)

Skills

- Base: React, Typescript, Vite
- Style: Styled-component
- 상태 관리: React Context API

[웹뷰] 엘리 앱

2022.09 - 2023.01

배달 라이더를 대상으로 오토바이 및 킥보드를 리스/렌탈 해주는 서비스를 개발하였습니다.

Work

- 리스 신청 기능 개발

Skills

- Base: React, Typescript, Vite
- Style: Styled-component
- 상태 관리: React Context API

[백오피스 어드민] 엘리 운영 관리 어드민

2022.08 - 2023.01

엘리 서비스를 운영 관리하는 어드민을 개발하였습니다.

Work

- 기기 GPS 추적 트랙커
- 운영 기기 및 반납 포트 지도 관제 시스템
- 작업구역 설정 기능 개발
- 기기 원격 제어 기능 개발
- B2B 업체용 별도 어드민 개발

- 대시보드 그래프 개발
- 마케팅(공지사항 & 배너) 관리 기능 개발
- 배너 및 기기 이미지 업로드
- 엑셀 다운로드
- 화면 구성 기획

Skills

- Base: React, Typescript, react-query, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API
- 지도: Naver Maps API
- Mock: MSW

[엘리 프로토타입]

2022.07 - 2022.08

배달 라이더를 위한 신규 서비스 기획 프로토타입

Work

- 배달 시장 조사 및 인터뷰
- 라이더 커뮤니티 사이트 개발 및 반응 조사
- 라이더 배달 랭킹 사이트 개발 및 반응 조사

Skills

- Base: React, Typescript, Vite
- Style: Styled-component
- 상태 관리: React Context API

[백오피스 어드민] 마케팅 관리 어드민

2022.07 - 2022.07

스윙 앱에 들어가는 공지사항, 배너, 팝업 마케팅을 관리하는 운영 어드민을 개발하였습니다.

Work

- 팝업 이미지 업로드 및 관리 기능 개발

Skills

- Base: React, Typescript, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API

[백오피스 어드민] 오늘은 라이더 어드민

2022.05 - 2022.09

오토바이 리스/렌탈 서비스인 오늘은 라이더를 운영하는 백오피스 어드민을 개발하였습니다.

Work

- 운영 기기 및 반납 포트 지도 관제 시스템
- 작업구역 설정 기능 개발
- 기기 원격 제어 기능 개발
- 마케팅(공지사항 & 배너) 관리 기능 개발

Skills

- Base: React, Typescript, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API
- 지도: Naver Maps API, Google Maps API, Kakao Maps API

[백오피스 어드민] 스윙 JP 어드민

2022.03 - 2022.08

스윙 글로벌 프로젝트 중 일본 서비스를 운영하는 백오피스 어드민을 개발하였습니다.

Work

- 운영 기기 및 반납 포트 지도 관제 시스템
- 작업구역 설정 기능 개발
- 국제화(국문, 일문)
- 매출, 라이딩 현황 대시보드 개발
- 마케팅(공지사항 & 배너) 관리 기능 개발

Skills

- Base: React, Typescript, Vite
- Style: Antd, Styled-component
- 상태 관리: React Context API
- 지도: Google Maps API
- 국제화: i18n

학력



송실대학교

2012.03 - 2018.08 | 졸업 | 의생명시스템학부

스킬

GitHub

HTML

JavaScript

NodeJS

React

TypeScript

Git

CSS

JIRA

Confluence

GraphQL

Web Socket

React Testing Library

Vite

Vitest

Google Maps

Apple 인증

Playwright

pnpm

Cursor

Figma

Claude

링크

🔗 포트폴리오

<https://dev-jio.notion.site/FE-Developer-32edf8461a6880c79144cc09d9cd1eab>

🔗 깃헙

<https://github.com/jihkyuo>